

海巡留言過濾機制

Threads AI Bot (保險視界 insurance_vision_) | 只推原創貼文，跳過別人貼文底下的留言 | 2026-08-22

1. 要解決的問題

Threads 的關鍵字搜尋結果裡混著兩種東西：**原創貼文**與**別人貼文底下的留言**。實測某一批 8 篇結果中有 5 篇是留言。

對海巡的目的來說，留言幾乎沒有價值：

- 留言的作者多半不是在**提出需求**，而是在**回應別人的討論**
- 在留言串裡再插一句話，能見度極低
- 同業承接客戶的動作大量發生在留言區，推播過來全是雜訊

2. 為什麼不能只看搜尋結果的畫面

實測結論：搜尋卡片的 DOM 分不出留言與原創。

兩者在搜尋頁上長得一模一樣——同樣的頭像、帳號、內文、時間戳，沒有任何欄位或 class 可以區分。想靠畫面判斷這條路是走不通的。

能區分的資訊在**貼文自己的頁面上**：那個頁面的 HTML 裡帶有 `"is_reply": true / false` 欄位。

關鍵限制：這個欄位只有「瀏覽器 session」拿得到。

用 `httpx` 帶著同一份 cookie 去抓同一個網址，回應裡就是不會有這個欄位。只有在已登入的 Playwright 瀏覽器 context 內用 `fetch(url, {credentials:'include'})` 去要，Threads 才會把這個欄位吐出來。

這也是為什麼這段判斷必須寫在瀏覽器裡跑，不能拉到 Python 層用 HTTP client 處理。

3. 實際怎麼做

`threads_scraper.py:369` `_mark_replies()` · 呼叫點 `threads_scraper.py:991`

1 挑出要查的貼文

只看 `age_hours <= 200` 的貼文。更舊的反正會被後面的時間窗口擋掉，查了是浪費流量。

2 在已登入的瀏覽器頁面內平行抓取

用 `page.evaluate()` 注入一段 JS，對每篇貼文的網址發 `fetch`，最多 6 條並行。

3 用正規式取出 is_reply

`/"is_reply"\s*:\s*(true|false)/` —— 不解析整份 HTML，直接抓那個欄位。

4 標記回 ScrapedPost

只有明確回傳 `true` 的才設 `p.is_reply = True`，並回傳被標記的篇數供診斷用。

```
const url = it.u
  ? `https://www.threads.com/@${it.u}/post/${it.sc}`
  : `https://www.threads.com/t/${it.sc}`;
const r = await fetch(url, {credentials:'include'}); // 必須帶 session
const t = await r.text();
const m = t.match(/"is_reply"\s*:\s*(true|false)/);
out[it.sc] = m ? (m[1] === 'true') : null;           // 抓不到就是 null
```

三個刻意的保守設計

情況	處理	理由
單篇抓不到 <code>is_reply</code> (回 <code>null</code>)	視為 非留言 ，照常推播	寧可漏放一則留言，也不要誤殺一篇真正的原創貼文——原創貼文才是海巡唯一的產出
整段 <code>_mark_replies</code> 拋錯	吞掉例外，全部視為非留言	這是 加分項而非必要條件 。它壞掉不該讓整次海巡失敗，頂多推播品質下降
貼文超過 200 小時	不查	後面的時間窗口 (預設 6 小時) 本來就會擋掉，查了純屬浪費

設計原則：`fail-open` 用在這裡，`fail-closed` 用在相關性判斷。

留言判斷失敗時放行 (`fail-open`)，因為誤殺原創的代價高於漏放留言。但 `is_patrol_worthy` 的相關性判斷相反——不確定就擋 (`fail-closed`)，因為推錯東西的代價高於少推一則。兩者方向不同是刻意的。

4. 過濾發生在哪一步

main.py:404 起 · Phase 2 過濾迴圈

抓回來的貼文先**依時間由新到舊排序**，再逐一跑過濾條件。順序是有意義的——便宜的檢查排前面，最貴的 AI 判斷排最後：

順序	條件	成本	不通過的處理
1	已看過 (去重)	本地 DB 查詢	直接跳過
2	內文少於 20 字	字串長度	直接跳過
3	<code>is_reply</code> 為真	已在抓取時算好	跳過，並記為已處理
4	時間無法判斷 (9999)	欄位比較	跳過 (不記為已處理)
5	超過時間窗口 (6 小時)	欄位比較	跳過 (不記為已處理)
6	<code>is_patrol_worthy</code> 相關性	呼叫 LLM	跳過，並記為已處理

為什麼留言要「記為已處理」，時間不符卻不記？

留言這個屬性**永遠不會變**，記下來以後就不必再判斷一次。但「太舊」「時間不明」是**當下狀態**——同一篇貼文在下一輪可能因為抓到更準的時間而通過，所以不能封死。

同理，相關性判斷要記，因為那要花一次 LLM 呼叫，同一篇不值得問第二次。

5. 怎麼確認它有在運作

每次海巡的 log 與 Telegram 診斷訊息都會帶上留言篇數：

[海巡] 找到 13 篇貼文（時間可判斷:11/13，最新:0h 最舊:33h，7日內:11 篇，其中留言:5 篇）

[海巡] 關鍵字「保費」無新貼文（舊:3 時間不明:2 留言:5 不相關:3，共:13）

解讀時要注意：「留言:N」只算「這一輪新出現」的留言。

去重（第 1 步）排在 is_reply 統計之前，所以上一輪已經看過並記錄的留言不會再被計入。數字偏小是正常的，不代表過濾失效。

手動驗證

```
GET /admin/test-search?keyword=保費&mode=recent
```

回傳每篇的 `age_hours` 與完整 log。注意這支端點**不做去重與 is_reply 過濾**，是給人看原始抓取結果用的——拿它跟實際推播的內容對照，就能看出過濾擋掉了哪些。

6. 已知限制

- **每篇要多發一次 HTTP 請求**。13 篇貼文就是 13 次額外 fetch。並行數壓在 6 是為了平衡速度與「看起來像不像真人」——真人不會一秒之內開六個貼文頁，這個數字未來若要調，只會往下調。
- 依賴 Threads 頁面裡的 **is_reply** 欄位名稱。Meta 改欄位名的話會靜默失效：正規式抓不到就回 `null`，全部視為非留言——不會報錯，只會發現留言又混進推播裡。這是要留意的靜默失效點。
- **必須在登入狀態下才有效**。未登入時貼文頁不帶這個欄位，等於整個機制停擺（而且同樣是靜默的）。

程式碼位置：`threads_scraper.py:369 _mark_replies()` · `main.py:404 Phase 2 過濾迴圈`

相關設定：`PATROL_MAX_AGE_HOURS = 6`（推播時間窗口） · `_mark_replies(max_age_hours=200)`（查詢範圍）